

ClimaSentinel

Rapport synthétique sur l'éco-conception logicielle



Selim ABOU LEILA, Rhita MOUMMADE, Begum SOZER, Nathan GERMANY, Xan DOYHENART, Anais ROBERT

ClimaSentinel est éco-conçu « by design » : le projet limite l'exécution aux besoins réels, réduit les appels API inutiles et organise les données pour éviter les scans et calculs redondants.

1. Synthèse exécutive

ClimaSentinel est un pipeline de données météorologiques conçu pour calculer des scores de basculement critiques pour un ensemble de villes. Notre approche de l'éco-conception est fondamentalement architecturale : au lieu d'ajouter un module d'« écoconception » séparé, nous avons fait des choix de structure précis. Les trois piliers sont clairs : **éviter de laisser l'infrastructure tourner en continu, ne collecter rigoureusement que les données utiles, et précalculer les résultats uniquement lorsque le Dashboard en a besoin.**

Fonctionnellement, le projet permet un suivi complet de la météo, de la qualité de l'air, des débits de rivière, des données historiques et des projections climatiques. Notre architecture repose sur GCP (Cloud Scheduler, Cloud Run Job, BigQuery, dbt) et utilise une couche Gold structurée spécifiquement pour les outils de visualisation. C'est cette architecture qui incarne nos principes de sobriété numérique :

- **Sobriété d'exécution** : le traitement s'effectue en batch quotidien. Une fois la tâche terminée, l'environnement Cloud Run Job s'éteint automatiquement, éliminant le coût et l'impact d'une machine allumée en permanence.
- **Sobriété réseau** : nous avons mis des gardes-fous sur les requêtes les plus gourmandes. Par exemple, nous conditionnons l'appel des débits de rivière aux seules villes concernées, et nous limitons les projections CMIP6, très volumineuses, à une seule mise à jour par mois.
- **Sobriété de la donnée** : nous optimisons le stockage et l'analyse. Les tables brutes sont partitionnées, la couche Silver utilise majoritairement des vues pour éviter toute duplication inutile, et la couche Gold est matérialisée en tables pour fournir des résultats pré-calculés et rapides au Dashboard.

En bref, ClimaSentinel réalise une double optimisation : celle de l'impact environnemental et celle du coût d'exploitation. Nous consommons moins de CPU, réduisons les appels réseau, évitons le stockage dupliqué et minimisons les requêtes BigQuery répétées. Il faut noter que cette analyse reste qualitative. Elle identifie les mécanismes de sobriété directement observables dans le code et la documentation, mais ne prétend pas être une mesure carbone complète et chiffrée.

Tableau 1 - Lecture rapide des leviers d'éco-conception identifiés.

Pilier	Choix observé	Effet attendu
Compute	Cloud Run Job + Scheduler quotidien, 1 vCPU, 512 Mo, 600 s	Moins d'infrastructure inactive et budget de calcul borné
Réseau/API	River API selon river_enabled et CMIP6 seulement le 1er du mois	Moins d'appels externes et moins de données transférées
Data	Raw partitionné, Silver en vues, et Gold en tables pré-calculées	Moins de stockage dupliqué et moins de scans/calculs dashboard

2. Sobriété de l'infrastructure et du calcul

Le premier atout de ClimaSentinel pour l'éco-conception, c'est l'exécution **serverless**. Le pipeline d'ingestion utilise un Cloud Run Job lancé par Cloud Scheduler. L'avantage est clair : c'est bien plus sobre qu'une machine virtuelle classique, car le serveur applicatif s'éteint complètement entre chaque exécution. Le code montre une cadence quotidienne à 6h00 UTC, avec des ressources conteneur très limitées : 1 vCPU, 512 Mo de mémoire et un temps maximum de 600 secondes.

- **Mise à l'échelle à zéro** : le job ne tourne que lorsqu'il est lancé. Entre deux exécutions, il n'y a aucune consommation de calcul pour l'ingestion.
- **Ressources encadrées** : limiter le CPU et la RAM empêche tout surdimensionnement. Le temps maximum (timeout) évite qu'un traitement qui plante ne tourne indéfiniment.
- **Traitement en lot (batch) quotidien** : Nous évitons l'interrogation constante (polling). Pour des données météo de pilotage, une mise à jour planifiée est toujours privilégiée par rapport à l'ingestion en temps réel, beaucoup plus gourmande.
- **Infrastructure fiable et reproductible** : L'usage de Terraform garantit l'absence d'écarts de configuration, pour une architecture simple, documentée et sous contrôle.

C'est l'essence même du Green IT : dimensionner le service en fonction du besoin réel. ClimaSentinel doit produire un score pour le tableau de bord, et non maintenir un service transactionnel continu. Le mode batch réduit donc la consommation de calcul au strict minimum nécessaire : lire la configuration, appeler Open-Meteo, insérer dans BigQuery, puis transformer avec dbt.

Tableau 2 - Effets concrets de la sobriété compute.

Mécanisme	Pourquoi c'est sobre	Indice dans le projet
Cloud Run Job	Pas besoin de machine virtuelle 24h/24, le service ne tourne que ponctuellement.	Architecture README + documentation ingestion
Scheduler 0 6 * * *	Une seule exécution par jour, ce qui élimine les boucles d'interrogation permanentes.	Terraform variables + README
1 vCPU / 512 Mi / 600 s	Le budget CPU, la mémoire et la durée sont strictement limités.	Terraform main.tf
dbt lancé dans le déploiement	Garantit que les transformations sont bien orchestrées, reproductibles et testées.	Makefile / README / docs dbt

3. Sobriété réseau et appels API

Le deuxième levier, c'est la frugalité de nos échanges avec le monde extérieur. Les données externes sont indispensables pour calculer nos scores, mais chaque requête pèse son poids : consommation réseau, parsing de fichiers JSON, et stockage dans BigQuery. Pour garder un système léger, ClimaSentinel applique quelques règles de bon sens fonctionnel.

- On ne travaille qu'avec l'essentiel : c'est un fichier de configuration qui pilote le système. Actuellement, seules les 10 villes européennes activées sont traitées, et on a même ajouté une option `river_enabled` pour ne pas solliciter les serveurs inutilement.
- L'API River Discharge, par exemple, ne s'active que si `river_enabled` est à "true". Dans notre configuration actuelle, cela cible uniquement Paris, Amsterdam et Varsovie, épargnant ainsi des appels inutiles pour toutes les autres villes.
- Pour les projections climatiques CMIP6, qui sont particulièrement lourdes (plus de 3 000 lignes par ville), on a mis en place un garde-fou : elles ne sont récupérées que le premier jour du mois. Le reste du temps, le système ignore simplement ces requêtes volumineuses.
- Enfin, on a limité la profondeur des données : on ne récupère que 7 jours de météo, 5 jours pour la qualité de l'air et une semaine glissante pour l'historique ERA5. On évite ainsi de télécharger des volumes de données sans fin qui n'auraient pas d'utilité immédiate.

En résumé, ClimaSentinel s'adapte à l'usage réel : on n'interroge pas tout le monde à la même fréquence. Si la météo et l'air changent vite et méritent un point quotidien, les projections à long terme sont mutualisées au mois. C'est cette gestion "intelligente" qui nous permet de réduire le trafic réseau sans jamais sacrifier la qualité des informations du dashboard.

4. Éco-conception de la donnée et de BigQuery

Le troisième pilier, c'est la façon dont on gère la donnée, un aspect souvent sous-estimé dans les projets data. L'impact environnemental ne vient pas juste de l'espace de stockage ; il vient surtout du volume de données qu'on doit **scanner** à chaque transformation et à chaque fois que quelqu'un consulte le tableau de bord. C'est pourquoi ClimaSentinel s'appuie sur une architecture "médaillon" (Bronze, Silver, Gold) pour bien séparer les rôles : les données brutes (raw) sont pour l'ingestion initiale, les données intermédiaires (stg) pour le nettoyage, et les données métiers (mart) pour le résultat final.

4.1. Des tables brutes bien organisées (partitionnement)

Pour éviter de lire toute l'histoire de nos données, le loader BigQuery partitionne les tables brutes (raw) par jour, soit sur l'horodatage `valid_ts_utc` pour les données horaires, soit sur la date pour le quotidien. C'est crucial : quand on lance une transformation, on ne vise que les partitions qui nous intéressent. En plus, on ajoute les champs `ingestion_run_id` et `ingested_at_utc` pour assurer la traçabilité et ne jamais dupliquer des lignes par erreur.

4.2. Un choix malin : Vues pour le Silver, Tables pour le Gold

Avec dbt, on a fait un choix d'économies. On matérialise les modèles intermédiaires (stg) comme de simples **vues**. Cela nous permet d'éviter de faire une copie physique de toutes les données nettoyées, gardant ainsi notre couche Silver ultra-légère. Par contre, on matérialise la couche finale (mart) en **tables**. Pourquoi ? Pour que les scores, classements et zones opérationnelles soient pré-calculés et prêts à l'emploi. Ce compromis est gagnant : on ne stocke rien d'inutile entre les étapes, mais on fournit des résultats rapides et stables au tableau de bord.

4.3. Pré-calculer pour le Dashboard

Le modèle `mart_city_score_history` est partitionné par date et s'occupe de calculer tous les sous-scores (chaleur, vent, pluie, air, rivière) avant de retenir le score de basculement global maximal (`global_tipping_score`). Ensuite, des vues comme `mart_city_score_current` et `mart_city_zone_current` simplifient l'accès pour le

dashboard. Le bénéfice est immédiat : l'utilisateur n'a pas besoin de relancer la logique métier lourde de calcul à chaque clic.

Tableau 3 - Compromis de matérialisation et sobriété.

Couche	Rôle	Matérialisation	Gain écologique / coût évité
Bronze / raw	Données API brutes, en mode ajout	Tables partitionnées	On scanne moins de données, tout en gardant une source auditable
Silver / stg	Nettoyage, déduplication, harmonisation	Vues	On évite de dupliquer toute la couche intermédiaire
Gold / mart	Score, classement, zones, drivers	Tables + vues finales	On a des résultats pré-calculés pour une consultation rapide (Power BI / Streamlit)

4.4. Usage sobre côté Power BI

Dans la documentation, on recommande le mode **Import** pour Power BI. C'est logique : les données de ClimaSentinel se mettent à jour une fois par jour. Avec ce mode, Power BI récupère les données à l'heure prévue. Si on utilisait le mode *DirectQuery*, chaque filtre ou clic d'utilisateur déclencherait une nouvelle requête BigQuery, générant ainsi du calcul et du trafic réseau totalement redondants.

5. Évaluation, limites et améliorations possibles

L'éco-conception de ClimaSentinel est solide pour un projet data cloud de niveau M1, car elle s'attaque directement aux principaux leviers de consommation : le calcul (*compute*), le réseau, le stockage et les requêtes analytiques. Le seul point à améliorer, c'est l'instrumentation : nous n'avons pas encore d'indicateurs environnementaux chiffrés pour mesurer précisément l'impact de ces choix.

Tableau 4 - Bilan synthétique.

Point évalué	Appréciation	Commentaire
Sobriété compute	Très bon	Serverless, batch quotidien, ressources limitées
Sobriété réseau	Très bon	Appels conditionnels et fréquence adaptée à la valeur de la donnée
Sobriété data	Bon	Partitionnement, vues Silver, Gold pré-calculée ; clustering à vérifier côté BigQuery
Mesure d'impact	À renforcer	Pas encore de suivi CO2, durée moyenne d'exécution, Go scannés ou appels économisés

Recommandations prioritaires

- Ajouter un court fichier docs/eco-conception.md listant les choix Green IT, les hypothèses et les métriques suivies.
- Suivre dans le temps : durée du Cloud Run Job, nombre d'appels API, lignes insérées, Go scannés par dbt et par Power BI, coût BigQuery mensuel.
- Vérifier ou ajouter le clustering par city_id pour les tables mart les plus filtrées, notamment si les usages dashboard augmentent.
- Conserver le mode Import dans Power BI et ajouter du cache côté Streamlit si un dashboard Python est utilisé.
- Réévaluer la matérialisation des vues Silver si elles sont enquêtées très fréquemment : une table incrémentale peut parfois consommer moins qu'une vue recalculée en boucle.

Conclusion

ClimaSentinel est éco-conçu par ses choix d'architecture plus que par un discours marketing : le système traite peu, au bon moment, avec des ressources limitées et des données organisées pour réduire les recalculs. La prochaine étape serait de rendre cette sobriété mesurable, avec quelques indicateurs opérationnels simples dans la documentation et dans les logs.

Sources consultées

[1] Dépôt GitHub ClimaSentinel - README et architecture -

<https://github.com/Selim-Abouleila/ClimaSentinel>

[2] docs/2-ingestion-pipeline.md - cadence, APIs, volumes journaliers -

<https://github.com/Selim-Abouleila/ClimaSentinel/blob/main/docs/2-ingestion-pipeline.md>

[3] infra/terraform/main.tf et variables.tf - Cloud Run Job, Scheduler, limites CPU/RAM/timeout -

<https://github.com/Selim-Abouleila/ClimaSentinel/tree/main/infra/terraform>

[4] ingest/main.py, fetcher.py, loader.py - conditions river_enabled, CMIP6 mensuel, partitions BigQuery -

<https://github.com/Selim-Abouleila/ClimaSentinel/tree/main/ingest>

[5] transform/dbt_project.yml et modèles mart/stg - matérialisation vues/tables et score -

<https://github.com/Selim-Abouleila/ClimaSentinel/tree/main/transform>

[6] docs/5-guide-powerbi.md - recommandation Import et rafraîchissement quotidien -

<https://github.com/Selim-Abouleila/ClimaSentinel/blob/main/docs/5-guide-powerbi.md>

[7] Clima Sentinel Cadrage Projet.pdf - document de cadrage fourni

L'idéation, la recherche documentaire, l'architecture du projet et l'analyse critique présentées dans ce document sont le fruit exclusif du travail de l'équipe. Des outils d'intelligence artificielle générative ont été utilisés en fin de processus uniquement comme assistants de rédaction pour structurer nos idées, formuler le texte de manière optimale et mettre en forme les tableaux comparatifs.